

Set 6 : <https://claude.ai/public/artifacts/83eee27a-0509-45c1-be08-cc3e8b4cbfc8>

Multiple features in your app each maintain their own separate conversation with Claude for the same end user, and none of them share any awareness of the others. A user complains the assistant 'forgets' things between features. What's the actual architecture answer here?



Every feature must always share one giant conversation so nothing is ever forgotten



This is always a bug that must be fixed by merging all conversations



This may be an intentional design tradeoff: some features genuinely warrant separate, isolated conversations, and cross-feature memory is a distinct feature to design deliberately if wanted, not an automatic default



The context window is too small and must be increased before anything else can be considered

Correct

Session boundaries are a design choice: isolated conversations per feature are often correct for scoping and privacy, and shared cross-feature memory, if wanted, needs to be built deliberately rather than assumed. Forcing one shared conversation ignores legitimate isolation needs, window size is not the driver here, and calling it an automatic bug skips the actual design tradeoff.

You want to forecast next quarter's Claude spend for a feature that's growing 15% month over month. What inputs does a sound cost model need?

- ☐ The size of the context window, regardless of how much of it is actually used per call
- ☐ The number of engineers on the team
- ☒ Historical token usage per request, the growth rate, and current per-token pricing, projected forward
- ☐ Only the current month's total invoice, extrapolated by guesswork

Correct

A cost model needs actual usage data, a growth assumption, and pricing, projected forward systematically. Team size is irrelevant to spend, a single invoice extrapolated by guesswork lacks a growth basis, and window size alone says nothing about tokens actually consumed.

Your application's schema for a structured extraction task has 40 optional fields, and the model frequently fills in plausible-looking but incorrect values for fields that weren't actually present in the source document. What design issue does this point to?

- ☐ The model tier is too small and should simply be upgraded
- ☐ The schema has too few fields and should be expanded further
- ☒ The schema likely needs a way to represent 'field not found' explicitly, rather than forcing a value into every field regardless of whether it's supported by the input
- ☐ The document input format is unsupported

Correct

Forcing a value into every field encourages fabrication when the source lacks that data; an explicit null or not-found representation lets the model report absence honestly. Model tier is not the driver of a schema design gap, the input format is not implicated, and adding more fields compounds rather than fixes the issue.

An agent has both a client-side tool (reads the user's current form input) and a server-side tool (queries a production database). A refactor accidentally swaps their execution locations. What breaks first?

☐ Nothing breaks, since tools are interchangeable regardless of where they execute

☐ The database tool becomes faster once running client-side

☒ The tool meant to read live client state can no longer see it once it's forced to run server-side, since the server has no visibility into the user's in-browser state

☐ Only latency changes; both tools still function identically otherwise

Correct

Client-side tools depend on access to the user's local browser state, which a server cannot see; swapping execution location breaks that access entirely. Tools are not interchangeable across execution boundaries, and a database query running client-side would typically fail or be insecure, not simply run faster.

Your instructions ask the model to 'be concise,' but responses stay long regardless. What refinement is more likely to work?



Switch to a different model tier without changing the instruction



Remove the instruction entirely, since vague adjectives never influence length at all



Replace the vague adjective with a specific constraint, such as a maximum sentence or word count



Repeat the word 'concise' several times in the same sentence for emphasis

Correct

A concrete constraint gives the model something specific to satisfy, unlike a vague adjective open to interpretation. Repetition of the same vague word does not add precision, dropping the instruction abandons the goal, and a tier swap does not fix an underspecified constraint.

A workflow currently branches on eleven different document types with a dedicated code path for each, and a twelfth type just appeared with no existing path. The team's instinct is to add a twelfth branch. What should you consider instead?



Immediately writing the twelfth branch, since workflows should always be extended to cover every new case as it appears



Deleting all eleven existing branches and starting over with no clear replacement



Whether the branching factor itself signals this is now agent territory, since new document types keep appearing faster than branches can be written



Ignoring the twelfth document type entirely until a branch is convenient to write

Not quite

A workflow whose branch count keeps growing faster than you can maintain is the signal to reconsider the architecture, not just add branch twelve. Endless branch-adding is the losing race that motivated the reconsideration, deleting everything with no plan is reckless, and ignoring new input types loses real work.

You're choosing a transport for an MCP server that will be called by a client on a different machine over the network, with no requirement for a persistent open connection between calls. Which fits best, and why not stdio?



A raw TCP socket is the only valid choice for any MCP server



An HTTP-based transport, since it supports request-based network communication; stdio assumes a local subprocess sharing standard input and output on the same machine



Either works identically regardless of whether the client and server share a machine



stdio, since it works over any network as long as both machines are on the same subnet

Correct

HTTP-based transport fits a remote, request-oriented client-server relationship; stdio is designed for a local subprocess communicating over shared standard input and output, not cross-machine networking. The two transports are not interchangeable regardless of topology, and MCP is not limited to raw TCP sockets as the only network option.

A team wants to add Claude-based text classification to an existing synchronous request-handling pipeline that currently blocks on every I/O call. What foundational concern should be addressed before adding the new blocking API call?

- ☐ Whether the team's code editor supports syntax highlighting for the new code
- ☐ Whether the git branch naming convention should change
- ☒ Whether the pipeline should move to asynchronous, non-blocking I/O so a slow classification call doesn't stall unrelated requests
- ☐ Whether the classification labels should be stored as strings or integers

Correct

Adding another blocking network call to an already-blocking pipeline is exactly the concurrency concern to address before shipping, since one slow call would otherwise stall unrelated work. Label storage format, branch naming, and editor tooling are unrelated to the concurrency risk being introduced.

A multi-step agent produces a wrong final report. The trace shows the retrieval tool returned the correct documents, and the model's summary of them is accurate, but a formatting tool downstream truncated the summary mid-sentence before it reached the user. Where is the fault?



In the formatting tool or its integration into the pipeline, since both retrieval and model reasoning were correct up to that point



In the retrieval tool, since it's the earliest step and therefore always the first thing to suspect



In the user's original request, since it must have been ambiguous



In the model's reasoning, since a wrong final report always originates from the model regardless of what the trace shows

Correct

Tracing shows the fault entering exactly where the truncation occurred, downstream of correct retrieval and correct model reasoning, so the formatting tool or its wiring is where to look. Blaming the model or the earliest step by default ignores what the trace actually shows, and nothing in the trace implicates the original request.

Your MCP server currently returns raw, unvalidated data straight from an upstream API to Claude, including occasional malformed records. What server-side practice would improve robustness?



Stop returning any data at all if the upstream service is ever imperfect



Validate and normalise the data before returning it, so malformed upstream records don't reach the model as if they were clean



Cache the malformed records so they are returned consistently every time



Pass everything through unchanged, since validation is solely the responsibility of whichever application calls the server

Correct

A well-built server validates and normalises what it returns, rather than passing upstream imperfections straight through as if clean. Deferring entirely to the calling application ignores the server's own responsibility, refusing all data over occasional flaws is an overcorrection, and caching malformed records preserves the very problem being raised.

Which TWO scenarios genuinely justify the Batches API over synchronous calls?

Select 2 · chosen 2

☐

Validating a form field as the user types



Re-indexing a document library overnight with no user waiting on the result

☐

Answering a customer's question in a live chat widget



Scoring a backlog of 200,000 support tickets for a weekly dashboard

Correct

Both are large, latency-tolerant jobs with no one waiting on an immediate response, the batch sweet spot. Live chat and as-you-type validation both need an immediate synchronous response.

A new model version is released with materially better reasoning but a documented change to how it escapes special characters in code output. Your pipeline parses that output with regex tuned to the old escaping. What is the safe migration path?

- ☐ Turn off the parser and pass raw output straight to users
- ☐ Ignore the new version permanently to avoid ever touching the parser again
- ☒ Test the new version against your parser and eval suite in staging, update the regex if needed, then promote deliberately
- ☐ Upgrade in production immediately, since better reasoning always outweighs a small formatting difference

Correct

A documented breaking change to output formatting demands validating and adjusting the dependent code before promotion. Upgrading blind risks silently breaking the parser, permanently freezing loses real capability gains, and disabling the parser abandons validation entirely.

**A requirement states 'the system must respond quickly.'
What is missing that would make this usable for design and testing?**

- ☐ The name of the engineer responsible for writing the code
- ☐ A list of every possible future feature the system might ever need
- ☐ Nothing; 'quickly' is specific enough to design and test against directly

☒ A concrete, measurable target, such as a specific latency percentile under a specific load, so it can be checked objectively

Correct

A vague adjective like 'quickly' cannot be checked without a concrete, measurable target such as a latency percentile under a defined load. It is not sufficiently specific as written, an exhaustive future feature list is unrelated to this gap, and naming an engineer does not make the requirement testable.

Which of these activities most clearly belongs to the design phase rather than the build or operate phases of the systems life cycle?

- ☐ Monitoring live cost dashboards after launch
- ☐ Fixing a production incident that occurred last night
- ☐ Writing the actual integration code against the chosen API
- ☒ Deciding the overall architecture, including workflow-versus-agent tradeoffs and model tier selection, before writing implementation code

Correct

Architectural decisions made before implementation begins are squarely design-phase work. Fixing an incident and monitoring dashboards are operate-phase activities, and writing the integration code is build-phase, both of which come after design decisions are made.

A single tool named `handle_request` currently does lookup, validation, and notification all in one call, and the agent frequently gets confused about what result it will get back. What is the most direct fix?

☐ Leave the tool as is but add a longer usage example in the system prompt

☒ Split it into three focused tools, each with one clear responsibility and a description that reflects it

☐ Rename the tool to something more exciting so it's memorable to the model

☐ Add a fourth responsibility so it's clearly the 'do everything' tool

Correct

Splitting an overloaded tool into focused, single-responsibility tools with matching descriptions removes the ambiguity about what each call returns. Renaming or adding more responsibilities does not address the confusion, and a longer example patches around a design problem rather than fixing it.

A user directly asks your assistant, in plain conversation, to roleplay as an unrestricted version of itself with no rules. Is this the same category of risk as a webpage with hidden malicious instructions, and does it need the same fix?

☐ No; the direct request is not a risk at all since the user is a legitimate, authenticated party

☐ Yes, but only because both involve text

☒ No; this is a jailbreak attempt via direct conversation, addressed through policy and refusal training, whereas the webpage case is injection via untrusted content, addressed through isolation and least privilege

☐ Yes; both are identical attacks solved by the exact same single technical control

Correct

These are genuinely distinct risk categories with distinct mitigations: jailbreak attempts are handled through the model's own policy and refusal behaviour, while content injection is handled through isolating untrusted data and limiting what it can trigger. Being authenticated does not make a jailbreak attempt harmless, and 'both involve text' is too shallow a similarity to imply one fix covers both.

A contractor's temporary access to your Claude API key needs to end automatically when their contract does, without anyone remembering to manually revoke it. What capability addresses this?



Increasing the rate limit on the key so contractor usage doesn't interfere with production traffic



Scoped, time-limited credentials or access grants tied to identity, rather than a long-lived shared key with indefinite validity



A stronger password on the shared key



Asking the contractor to delete the key from their notes when they leave

Correct

Time-bound, identity-scoped credentials expire on their own, removing the dependence on someone remembering to revoke access manually. A stronger password does not add an expiry, relying on the departing contractor to self-revoke is not a control, and rate limits address throughput, not access lifecycle.

A document-processing agent reads uploaded files from anonymous users before any human review. Which TWO practices reduce the risk of those files carrying manipulative embedded instructions?

Select 2 • chosen 2

☐

Increase the model's context window so it can read the whole file at once



Treat uploaded file content as untrusted data, isolated from the agent's own instructions



Scope the agent's available actions tightly, so even a manipulated read cannot trigger a sensitive operation

☐

Trust the content fully once the file passes a virus scan, since malware and prompt injection are the same category of risk

Correct

Isolating untrusted file content from trusted instructions, plus tightly scoping available actions, are the two levers that blunt embedded manipulation. A virus scan addresses malware, not textual injection, which is a different risk entirely, and context window size has no bearing on injection resistance.

A capability must call a live, authenticated internal API and also needs to be reachable identically from three unrelated internal tools your company builds. Weighing Skills, custom tools, and MCP, which fits, and what rules out a Skill here?



A Skill, since Skills can natively perform any authenticated API call given the right description



An MCP server; a Skill is instructional content and cannot itself make authenticated live API calls on its own



None of the three approaches fit a live authenticated API



A custom tool duplicated by hand into each of the three consuming tools

Correct

Live, authenticated API access reused across multiple consumers is the MCP server case; a Skill is packaged instructions and reference material, not a mechanism for making live authenticated calls itself. Hand-duplicating a custom tool three times creates the exact maintenance burden a shared server avoids, and this is squarely within scope for one of the three approaches.

You're deciding whether to build a custom agent loop from scratch or use the Claude Agent SDK. Your team has no existing orchestration code and a tight deadline. What does the SDK offer that changes the calculus?



Guaranteed lower per-token pricing versus calling the API directly



Built-in handling for the tool-use loop, context management, and common agent patterns, reducing what you must build and test yourself



Automatic conversion of any workflow into an agent regardless of whether that fits the task



A completely different model with different capabilities than the raw API

Correct

The Agent SDK provides scaffolding for the loop and common patterns, cutting the amount of custom orchestration code a tight deadline would otherwise require. It is the same underlying model family, does not change per-token pricing, and does not override the workflow-versus-agent decision for you.

A moderation feature must flag live chat messages in under a second, and separately re-score the entire message archive once a month for a compliance report. How should these two needs be split?

- ☐ Synchronous calls for both, accepting the higher cost on the monthly re-score to avoid maintaining two code paths
- ☐ Batches API for both, since consolidating everything onto one API path simplifies the codebase
- ☐ Streaming for the monthly re-score, since streaming is the cheapest option regardless of urgency
- ☒ Synchronous calls for live moderation; the Batches API for the monthly archive re-score

Correct

Each workload matches its own SLA: live moderation needs realtime response, and the tolerant monthly job belongs on Batches for cost. Consolidating either way sacrifices either latency or cost for no real benefit, and streaming does not reduce per-token cost.

An agent's context contains the full text of 12 API responses from earlier tool calls, most of which are no longer relevant to the current step, and quality is degrading. What is the appropriate context management move?

- ☐ Switch to a smaller model so the existing context fits more comfortably
- ☐ Add a 13th tool call to compensate
- ☐ Leave all 12 in place since removing any tool output risks losing information the agent might need eventually
- ☒ Prune the stale tool outputs, keeping only what remains relevant to the current step, to reclaim budget and reduce noise

Correct

Pruning stale, no-longer-relevant tool outputs is the direct fix for both budget pressure and quality degradation from noise. Keeping everything indefinitely is what caused the problem, adding another call adds more clutter, and a smaller model does not resolve an oversized, noisy context.

A tone-of-voice instruction sits in the system prompt, and a conflicting tone instruction appears in the user's message. Which generally takes precedence, and why does this matter for design?

- ☐ The user message always overrides the system prompt with no exceptions in any case
- ☐ Neither takes precedence; the model picks arbitrarily and this is unpredictable by design
- ☒ The system prompt is the higher-priority, more durable placement, so critical constraints belong there rather than relying on user messages to enforce them
- ☐ Whichever instruction is longer wins

Correct

The system prompt carries durable priority, which is exactly why critical constraints belong there rather than being left to a user message to reinforce. Length is not the deciding factor, and precedence is not arbitrary, it follows deliberate placement.

An error handler currently catches every exception type identically and always tells the user to 'try again later,' whether the cause is a bad user input, a rate limit, or a permanent server error. What is the improvement?



Keep the single generic message, since users don't need to know the distinction between error types



Classify errors by type and recoverability, and choose the recovery strategy and user-facing message per category rather than one generic response



Remove error handling entirely and let exceptions surface raw to the user



Log the errors in more detail internally without changing what the user is ever told

Correct

Different error categories call for different recovery strategies and different user messaging, which a single blanket response cannot provide. Users benefit from knowing whether to retry, fix input, or wait, better internal logging alone doesn't change the poor user experience, and removing handling entirely is a regression, not an improvement.

Your team is deciding where to place a content-policy check: only once, at the very end after the full response is generated, or additionally earlier in the pipeline. What does layering suggest?

- ☐ A single end-of-pipeline check is always sufficient and additional checks are pure redundancy with no value
- ☐ Policy checks should only ever run on user input, never on the model's own output
- ☐ Policy checks are unnecessary if the system prompt already states the policy
- ☒ Check at multiple points, since an end-only check misses the chance to prevent an issue earlier and provides no depth if that single check is bypassed or misses something

Correct

Layered guardrails at multiple points reduce reliance on any single check catching everything. A lone end check has no backup if it fails, restricting checks to input-only ignores output-side risk entirely, and a system-prompt statement alone is a probabilistic control, not a full guardrail.

An agent's memory currently holds every tool call and result from the entire session with no pruning, and by turn 40 the agent starts repeating earlier mistakes it should have already learned from. What does this suggest about how memory is being used?

- ☐ The agent needs a larger context window and nothing else needs to change
- ☐ Memory should be disabled entirely, since any memory clearly makes agents worse over time
- ☒ Raw accumulation without summarisation or structuring may bury the useful signal in volume, so what's kept needs curating, not just retained indefinitely
- ☐ The problem is unrelated to memory and is purely a model-capability issue

Correct

Unstructured accumulation can bury useful signal under volume; the fix is curating what's retained rather than keeping everything indefinitely or dropping memory altogether. A bigger window without curation still drowns in the same noise, and this is a memory-management issue, not simply a capability ceiling.